

EinsteinVision: Tesla-Style 3D Driving Visualization using 2 late days

Shibani Senthilbabu
RBE Department
Worcester Polytechnic Institute
ssenthilbabu@wpi.edu

Ajay Adit Jagan
RBE Department
Worcester Polytechnic Institute
ajagan@wpi.edu

Abstract—We present a Tesla-style 3D driving visualization system that takes dashcam video from a Tesla Model S and produces a rendered Blender scene showing the ego vehicle’s perception of the world. The pipeline processes 13 sequences end-to-end, covering object detection and tracking, metric depth estimation, lane detection, vehicle classification and 3D orientation, brake light and indicator detection, and parked/moving classification. All detections are rendered frame-by-frame in Blender with appropriate 3D assets.

I. PHASE 1: CORE DETECTION AND RENDERING

A. Object Detection and Tracking

The backbone of the pipeline is YOLO11m [1] pretrained on COCO, which runs on every frame of the front camera video. ByteTrack [2] provides persistent track IDs across frames, which is critical for the temporal smoothing and gap-filling that happens later in the pipeline. COCO class IDs are mapped to our labels: cars, buses, and trucks all become “vehicle”, while bicycles and motorcycles keep their subtypes for asset selection later. We apply per-class confidence thresholds (0.45 for vehicles, 0.30 for traffic lights, 0.55 for stop signs) tuned from the score distributions on our data.

Traffic light color classification works by splitting the bbox into vertical thirds and finding which third has the highest mean V-channel value among pixels with saturation above 50. Since the lit bulb is always the brightest, this simple spatial heuristic is reliable. We reject traffic light detections in the lower 40% of the frame (hood reflections) and boxes that are wider than they are tall (misdetctions).

Stop signs are filtered by position (upper 70% of frame only) and aspect ratio (roughly square), which eliminates most false positives from red tail lights.

Pedestrians whose bounding boxes overlap significantly with a bicycle or motorcycle (>30% of the bike’s area) are flagged as riders and excluded from pedestrian rendering to avoid placing both a person and a bike at the same spot.

B. Depth Estimation

Metric depth comes from Video-Depth-Anything-Large [3], which outputs per-pixel depth in meters using the camera’s focal length for scale. The output depth maps are saved as numpy arrays and looked up during rendering to position objects in 3D.

C. Lane Detection

Lane markings are detected using CLERNet [4], a transformer-based lane detector trained on the CULane dataset. Our frames are 1280×960 but CULane expects 1640×590 , so we crop 590 pixels from the vertical center of each frame and pad horizontally to 1640 pixels. After inference, the predicted lane points are mapped back to the original pixel coordinate space. Additionally BDD100K dataset was used as the fine-tuning dataset to improve CLERNet’s generalization from European highways to US road markings because this dataset is from Berkeley, USA.

Each lane is then classified by color and type. For color, we sample HSV values along the lane points and check whether the hue falls in the yellow range. For type (solid vs. dashed), we look at the brightness profile along the polyline — a dashed lane produces periodic bright/dark alternation, while a solid lane has relatively uniform brightness.

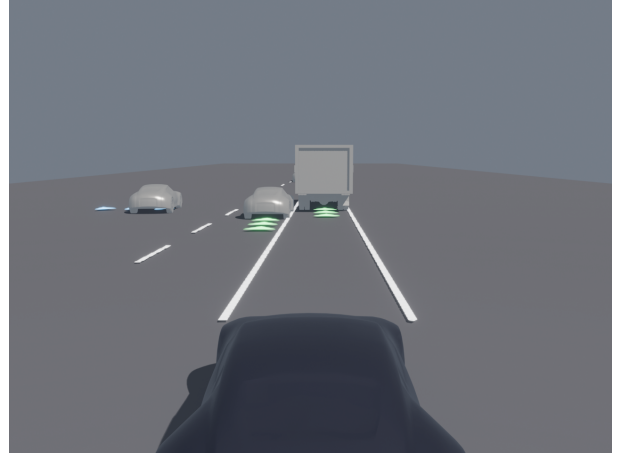


Fig. 1. Robust Lane Detection - Upto 4 lanes

D. Camera Calibration

Camera intrinsics were obtained by running OpenCV’s checkerboard calibration on the provided calibration videos. Only the front camera’s intrinsic matrix K is used in the pipeline since all detection and rendering operates on the front camera.



Fig. 2. Lane detection with lane classification for dashed and yellow lanes

E. Rendering

The Blender rendering script loads a pre-built base scene containing the ego vehicle, a road surface, and a camera, then populates it with 3D objects for each frame based on the detections. Objects are positioned by back-projecting their 2D bounding box centers into 3D using the camera intrinsics and depth map. When FCOS3D provides a 3D position for a vehicle, that takes priority over depth-map-based positioning since it tends to be more accurate.

Each object type gets its own visual treatment in the scene:

- **Vehicles** are loaded from per-subclass `.blend` files (sedan, SUV, hatchback, pickup, truck, motorcycle, bicycle). Each asset has a per-model yaw offset that we determined empirically because each 3D model faces a different direction at rest. Moving vehicles get a metallic paint material and a set of colored chevron arrows on the ground in front of them (green for same-direction, red for oncoming, blue for lateral). Parked vehicles are rendered in translucent gray without arrows.
- **Pedestrians** use a rigged mesh whose facing direction is set from the 3D shoulder keypoints estimated by RTMPose3D.
- **Traffic lights** are built procedurally as a dark rectangular box with three spheres (red, yellow, green). The active color is lit with emission; the rest are dark. For arrow-type traffic lights, a flat arrow mesh is placed behind the lit sphere.
- **Stop signs** use the provided asset with an applied octagonal texture, rotated to face the ego vehicle.
- **Speed limit signs** are procedural: a pole, white rectangular face, “SPEED LIMIT” text, and the detected numeric value, all oriented toward the ego car.
- **Road arrows** are flat white arrow meshes placed directly on the ground plane at their detected world coordinates.
- **Lanes** are projected from pixel coordinates to the ground plane using the camera intrinsics and extrinsics, then rendered as flat ribbon meshes (0.15m wide). Dashed lanes use a dash-gap pattern. Lane endpoints are extrapolated

olated toward the ego vehicle and the horizon for visual continuity.

- **Brake lights** are emissive red planes attached to the rear of vehicle meshes. A bloom compositor pass using Blender’s Glare (Fog Glow) node makes them glow realistically onto the car body.
- **Cones, cylinders, and dustbins** are loaded from the provided assets and placed at their depth-estimated positions.

II. PHASE 2: CLASSIFICATION AND 3D ESTIMATION

A. Vehicle Sub-Classification

YOLO’s COCO classes already distinguish bicycles, motorcycles, and trucks at detection time. For the remaining vehicles detected as COCO “car”, we run YOLO11m-cls pretrained on ImageNet on each vehicle crop and check if any of the top-5 predictions map to one of our asset types via a lookup table (e.g., ImageNet “sports car” → sedan, “jeep” → SUV, “pickup truck” → pickup). When none of the top-5 are vehicle classes, we fall back to a height-based heuristic: the real-world height is estimated from the bbox pixel height and the FCOS3D depth ($H_{\text{real}} = h_{\text{px}} \cdot Z / f_y$), and thresholded into sedan ($<1.55\text{m}$), SUV ($<2.0\text{m}$), or truck. Classifications are merged into the main detections file and smoothed per track via majority vote to prevent the same car from flickering between sedan and SUV across frames.



Fig. 3. Sample of Sub-Vehicle Classifications

B. Vehicle Heading

FCOS3D [5], a monocular 3D detector pretrained on nuScenes, provides the yaw angle, metric depth, and 3D bounding box dimensions for each detection. We match FCOS3D outputs to YOLO detections by 2D IoU (threshold 0.15) and write the matched yaw into each detection. The FCOS3D yaw uses camera coordinates where 0 means facing right (+X) and $\pi/2$ means facing toward the camera ($-Z$), so we convert to our Blender convention (0 = away from ego, π = toward ego) via $h = \text{yaw} - 3\pi/2$, normalized to $[-\pi, \pi]$.

Since FCOS3D doesn’t fire on every frame for every vehicle, we do several post-processing steps. A sliding median

window of 5 frames smooths noisy yaw values per track. Frames where FCOS3D didn't match a tracked vehicle get the heading propagated from the nearest frame in the same track that did have a match. Gaps of up to 10 frames where a tracked vehicle temporarily disappeared are filled by linearly interpolating the bbox position and depth from the surrounding frames.

C. Traffic Light Arrows

To classify whether a traffic light has a directional arrow, we first extract the lit bulb region from the relevant third of the bbox (top for red, middle for yellow, bottom for green) using per-color HSV thresholds. For red, that means $H \in [0, 12] \cup [165, 180]$ with $S > 20$ and $V > 50$. Morphological close and open with a 3×3 elliptical kernel clean up the mask.

The primary classification method is shape analysis on this thresholded mask. Five features are computed: circularity ($4\pi A/P^2$), solidity (A/A_{hull}), the number of significant convexity defects (those with depth exceeding 3% of the contour perimeter), Harris corner count, and the number of vertices from Douglas-Peucker polygon approximation ($\epsilon = 0.03 \cdot P$). Each feature casts a weighted vote — low circularity (< 0.45) contributes 0.4 toward “arrow”, while convexity defects contribute 0.1–0.2 depending on count. A detection is classified as an arrow when the total score reaches 0.45 and at least one convexity defect is present.

When the shape is classified as an arrow, its direction is determined by running PCA on the contour points. The tip of the arrow is the extreme end of the principal axis that has fewer nearby contour points (within 20% of the total projection range). If the eigenvalue ratio is below 1.5 (nearly isotropic shape), direction defaults to “straight” since there isn't a clear elongation axis. Otherwise, the principal axis angle is binned into left, right, or straight.

As a fallback when shape analysis is inconclusive, the mask is resized and matched against procedurally generated arrow and circle templates using `TM_CCOEFF_NORMED`. An arrow is accepted only when the best arrow template score exceeds the circle template score by more than 0.08 and clears 0.15 in absolute terms. All results are smoothed per traffic light track via majority vote.

D. Speed Limit Signs

Speed limit detection uses a two-stage approach. YOLO-World [10] with the open-vocabulary prompt “road sign” generates candidate bounding boxes at a low confidence threshold (0.10), filtered to the upper 70% of the frame and a minimum size. Each candidate crop is padded by 10% and, if smaller than 60×50 pixels, upscaled for better OCR performance. EasyOCR then reads the crop with an allowlist restricted to digits and the letters in “SPEED LIMIT”. OCR is run twice (once on the original crop and once on an adaptive-thresholded binary version) and the highest-confidence result that parses to a valid US speed value is kept (5, 10, 15, ..., 75 mph). Detections below 0.45 confidence are discarded, and values



Fig. 4. Traffic Arrow Identification

are smoothed per track. In Blender, a procedural speed sign asset is placed with the detected number.

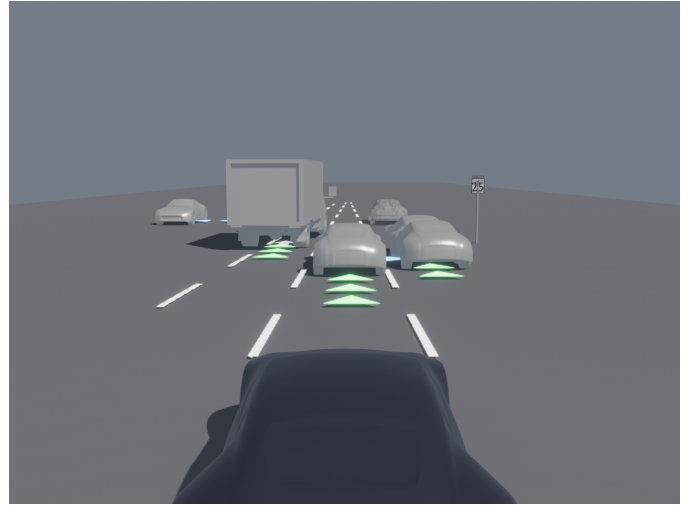


Fig. 5. Speed Limit Sign

E. Road Ground Arrows

Arrows painted on the road surface are detected using classical computer vision rather than a learned model. A bird's-eye-view homography is computed from the front camera intrinsics and extrinsics, mapping a world-space rectangle ($\pm 4\text{m}$ lateral, 10–40m forward) onto a 400×600 pixel top-down view. This BEV image is converted to grayscale and adaptive-thresholded (Gaussian, block size 51, $C=-8$) to isolate white paint markings, followed by morphological close and open to clean up noise.

Contours in the thresholded image are filtered by area (400–15000 px), aspect ratio (< 3), minimum size (25×25 px), and solidity (< 0.90 , since arrows have concavities that circles and rectangles don't). Each surviving contour is padded to



Fig. 6. Stop Sign

a square, resized to 40×40 , and matched against procedural arrow templates (straight, left, right) and a circle template. An arrow is accepted when its best template score exceeds the circle score by > 0.10 and clears 0.55 absolute. The detected world position is computed from the BEV centroid, filtered to $\pm 3.5\text{m}$ of the ego lane center, and projected back to an image-space bounding box via the inverse homography. Arrows are tracked across frames by world-space proximity ($< 4\text{m}$) and direction is smoothed by majority vote.

F. Additional Objects

Traffic cones, cylinders, poles, and dustbins aren't covered by COCO, so we detect them using Detic [6] with a custom 6-word vocabulary ("cone", "traffic cone", "traffic cylinder", "traffic pole", "dustbin", "barrel").

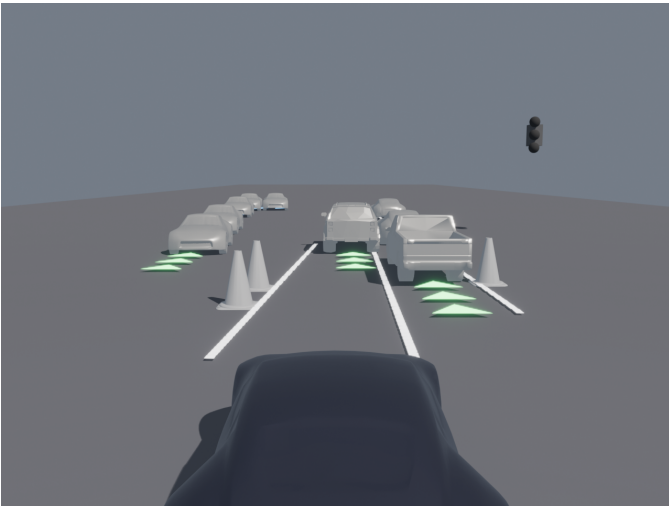


Fig. 7. Sample of Additional objects Detection

G. Pedestrian Pose

Pedestrian facing direction and pose are estimated using RTMPose3D (RTMW3D-L) [8] from MMPose, which outputs 133-point whole-body 3D keypoints. The RTMDet person detector finds bounding boxes, then RTMPose3D estimates 3D keypoints for each. We match these to our tracked pedestrian detections by IoU, with a nearest-center fallback when IoU matching fails.

The facing direction is derived from the 3D shoulder line: the perpendicular to the left-right shoulder vector in the XY plane gives the direction the person's chest is pointing. This angle is applied as the Z-rotation of the pedestrian mesh in Blender.



Fig. 8. Brake Light Detection

III. PHASE 3: BRAKE LIGHTS AND MOTION

A. Brake Light Detection

Instead of color thresholding on a fixed crop region, we use Detic with the full LVIS-IN21k vocabulary, which has "taillight", "brake_light", and "blinker" as real class labels. A single Detic forward pass per frame finds all rear-light bounding boxes. Each light detection is matched to a tracked vehicle by containment, and we measure red-brightness in a strip region around each taillight bbox with the taillight bbox itself masked out. The always-on taillight produces a steady glow in its bbox, but when the brake fires, there's additional red spillover into the surrounding housing, reflectors, and bumper that isn't there during normal driving.

The raw per-frame strip scores go through a combined post-processing strategy. One branch uses a delta-hold detector: it fires when the strip score jumps significantly from the previous frame and holds the "braking" state for up to 30 frames or until a symmetric drop is detected. A motion filter then suppresses flags for vehicles whose bbox height is not growing over a local window, since a non-growing bbox means the vehicle is pulling away or matching ego speed and therefore not braking relative to us. The other branch uses a per-track bulb brightness

baseline: it computes the median $V \geq 200$ fraction inside each vehicle’s taillight bbox over the track’s history and flags frames where brightness exceeds $1.5 \times$ the median or the median plus 15% absolute. This handles the case where both vehicles are stopped (the motion filter would wrongly suppress the delta-hold branch). The final brake signal is the union of both branches.

B. Parked vs. Moving

Vehicle motion is classified using RAFT [9] optical flow. For each pair of consecutive frames, RAFT computes dense flow with 20 iterations. A global homography is estimated via RANSAC on subsampled point correspondences (every 10th pixel) to factor out ego-motion. The residual flow — what’s left after subtracting the homography-predicted background flow — captures independent object motion. Vehicles with median residual magnitude above 0.2 pixels inside their bbox are classified as moving.

For moving vehicles, we also estimate a travel direction. The mean residual flow vector within the bbox is back-projected to a 3D velocity using the camera focal length and an estimated depth from bbox height ($Z = f_x \cdot 1.5 / h_{\text{bbox}}$). Direction classification prioritizes lane geometry when available: vehicles to the left of the yellow center divider are labeled “oncoming”, and the rest default to “ahead”. When no lane data is present, we fall back to the sign of the vertical flow component. The resulting labels are passed to the renderer, which applies the visual distinction described in Section I-F.



Fig. 9. Visualization of vehicle motion estimation using directional flow vectors. **Red chevrons** indicate oncoming traffic, **green chevrons** represent vehicles traveling in the same direction as the ego-vehicle, and **blue chevrons** denote transverse motion (left or right). The orientation of each chevron points in the instantaneous direction of the vehicle’s heading.

IV. CORNER CASES AND LIMITATIONS

Night scenes are the biggest challenge across the board. Depth estimation becomes less reliable, and streetlights produce amber reflections on car surfaces that trigger false indicator detections. The per-track bulb brightness baseline

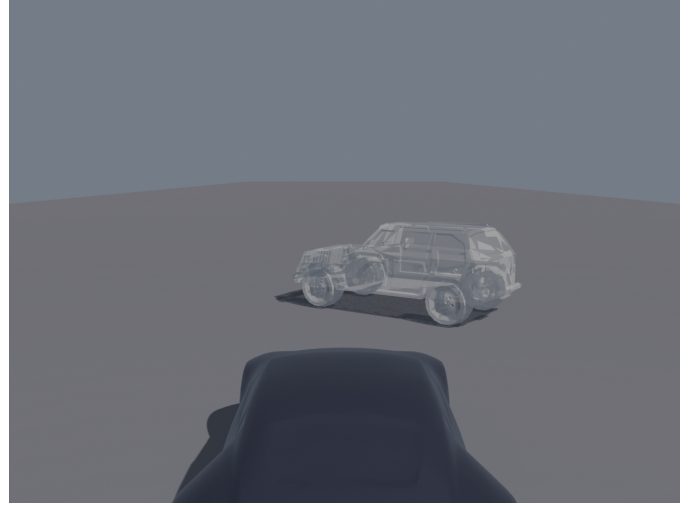


Fig. 10. Parked Vehicle Differentiation

helps with brake detection at night, but overall the system produces noisier output after dark.

FCOS3D matches roughly 60–70% of YOLO detections. The remaining vehicles only get optical flow heading, which can distinguish same-direction from oncoming but can’t estimate lateral angles. Rather than rendering these at potentially wrong depths, we skip them entirely.

Small and distant vehicles are a persistent weak point. The bounding boxes are too small for reliable sub-classification, heading estimation, or brake light detection. CLRRerNet also struggles with lanes that are partially occluded by vehicles or washed out by glare, and since the lane and object detectors run independently, there’s no occlusion reasoning between them.

The BEV homography for road arrow detection assumes a flat road surface. On hilly or curved sections, the perspective transform distorts arrow shapes enough that template matching accuracy drops. Similarly, EasyOCR occasionally misreads digits on small or angled speed limit signs — per-track majority vote catches most of these, but an isolated single-frame sign can still be wrong.

Our current system utilizes YOLO World for its open-vocabulary capabilities, allowing for flexible detection without custom training. However, this lack of domain-specific fine-tuning leads to occasional false positives, where various road signs are misidentified as speed limits due to shared visual features (e.g., shape and numerical content).

While many SOTA lane detection models perform well in their original environments, they often struggle to generalize to US road conditions because they were primarily trained on European data. To bridge this gap, we fine-tuned our model on the BDD100K dataset from Berkeley. This allowed the system to adapt to the specific markings and environmental variables found in US driving scenarios, significantly improving its robustness.

V. POSSIBLE IMPROVEMENTS

Using a multi-camera setup (side repeaters and rear camera) would improve coverage for vehicles entering or leaving the frame laterally. A learned brake light detector fine-tuned on labeled brake/no-brake pairs would likely be more robust than our Detic + strip-masking heuristic. Better datasets would improve lane detection and vehicle subclassification.

VI. CHALLENGES

Getting FCOS3D’s yaw convention right was surprisingly tricky. FCOS3D uses camera coordinates where yaw 0 means the vehicle’s longitudinal axis is along the camera’s +X (i.e., sideways in the image), while our renderer expects 0 to mean “facing away from ego”. An off-by- π error makes every single vehicle face the wrong direction, and the bug is hard to spot because the rendering still looks plausible — just with all oncoming traffic shown as same-direction and vice versa.

Our first brake light detector used a fixed V-channel brightness threshold, which over-fired at night (taillights are always bright against a dark background) and under-fired during the day (brake lights are dim compared to ambient sunlight). Switching to a per-track adaptive baseline that compares each vehicle against its own brightness history partially solved both failure modes.

Each Blender vehicle model faces a different direction at yaw=0 in its rest pose (sedan faces toward ego, SUV faces away, pickup is stored on its side and needs uprighting). We had to determine per-asset yaw offsets by rendering each model at multiple rotations and checking which one looked correct.

While many SOTA lane detection models perform well in their original environments, they often struggle to generalize to US road conditions because they were primarily trained on European data. To bridge this gap, we fine-tuned our model on the BDD100K dataset from Berkeley. This allowed the system to adapt to the specific markings and environmental variables found in US driving scenarios, significantly improving its robustness.

In Phase 2, one of the biggest challenges in the traffic arrow classification task was the extremely small area of interest. The traffic light itself occupied only a tiny portion of the image, and the individual arrow bulb was even smaller, which made reliable classification difficult from the start.

For motion classification, we used RAFT optical flow between consecutive frames and initially computed Sampson distance to identify independently moving vehicles. However, Sampson distance only captures motion that violates epipolar geometry cars traveling in the same direction as the ego vehicle at similar speeds produce near-zero Sampson scores, making them indistinguishable from parked cars. We addressed this by switching to a homography-based residual flow approach, where we fit a homography to the dominant ground plane and subtract the predicted ego-motion flow from the actual flow. The residual magnitude reliably separates moving from static vehicles regardless of relative direction.

Classifying the direction of motion proved equally challenging the residual flow strips out the approach/recede component (since the homography absorbs it), leaving primarily lateral motion even for oncoming cars. Furthermore, a systematic positive vertical bias in the residual — caused by vehicles being 3D objects above the flat road plane assumed by the homography — made threshold-based classification fragile.

After iterating through several approaches including raw flow direction, physics-based 3D velocity backprojection using depth from bounding box height and camera intrinsics, we arrived at a hybrid solution: the detected lane divider (yellow solid or leftmost dashed white line) serves as the primary classifier — vehicles to its left are oncoming, vehicles to its right are traveling ahead — with flow-based velocity as a fallback when lane data is unavailable.

VII. CONTRIBUTION

Shibani: Detection pipeline, depth estimation, FCOS3D heading, vehicle classification, Detic object detection, brake light detection, pedestrian pose, rendering, documentation.

Ajay: Detection pipeline, lane detection (CLRerNet), traffic light arrow classification, speed limit sign detection, road arrow detection, motion classification (RAFT), rendering, documentation.

REFERENCES

- [1] Ultralytics, “YOLO11,” 2024. <https://docs.ultralytics.com/models/yolo11/>
- [2] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” *ECCV*, 2022.
- [3] C. Yang et al., “Video Depth Anything,” 2024. <https://github.com/DepthAnything/Video-Depth-Anything>
- [4] H. Honda and Y. Uchida, “CLRerNet: Improving confidence of lane detection with LaneIoU,” *WACV*, 2024.
- [5] T. Wang et al., “FCOS3D: Fully convolutional one-stage monocular 3D object detection,” *ICCVW*, 2021.
- [6] X. Zhou et al., “Detecting twenty-thousand classes using image-level supervision,” *ECCV*, 2022.
- [7] A. Radford et al., “Learning transferable visual models from natural language supervision,” *ICML*, 2021.
- [8] T. Jiang et al., “RTMPose: Real-time multi-person pose estimation based on MMPose,” 2023.
- [9] Z. Teed and J. Deng, “RAFT: Recurrent all-pairs field transforms for optical flow,” *ECCV*, 2020.
- [10] T. Cheng et al., “YOLO-World: Real-time open-vocabulary object detection,” *CVPR*, 2024.